

Acceptance Before Export: A Bounded Evidence Contract for Governance Telemetry

Imran Siddique

OPAQUE Systems

October 1, 2026

Technical report, version 3

Scope of this edition. This is an author-prepared technical report, not peer reviewed. Version 3 (October 1, 2026) corrects version 2 (September 30, 2026), which left out a defect in the pinned revision's TRACE finalizer that the paper's truncation argument depends on, and the Internet-Draft that already chains agent audit records over RFC 8785 bytes; results are unchanged. Version 2 added one prior-art citation that version 1 (September 27, 2026) omitted. Version 1 revised a manuscript dated September 4, 2026. Every result reported here is software-only: the Python test suite, the portable conformance fixtures and separate review probes, run on one Windows workstation against a pinned source revision, rerun on September 27, 2026, and compared with a later revision. No durable store, crash-restart, multi-process or deployed-exporter experiment was run.

Patent Pending.

Abstract

An operational telemetry backend can lose events through sampling, retention and exporter failures. Its observations therefore cannot, by themselves, establish complete governance evidence. We describe a source-side acceptance boundary in AgenTrust Telemetry: validate a metadata-constrained event, check context correlation, accept an isolated representation into a run-scoped cryptographic prefix, and only then attempt operational projections. In a durable configuration, an external callback must acknowledge persistence before the local prefix advances. Export failures remain distinct from evidence failures. We formalize the conditional ordering property and test its limits, including acknowledgment loss and an empty accumulator declared complete. At a pinned source revision, 111 Python tests and 13 portable conformance fixtures produced expected results on September 4 and again on September 27, 2026. Four additional probes demonstrate both the useful separation and the absence of automatic completeness or distributed exactly-once persistence. The contribution is a precise contract for acceptance before export, with explicit trust and recovery obligations, rather than a claim that a hash chain proves every event occurred.

1 Motivation and related systems

Agent runs cross policy engines, approval services, tools and asynchronous execution boundaries. The AgenTrust contract represents policy decisions, approval lifecycle, usage, classified data flow, action execution and evidence lifecycle as six event families. A durable run identifier and, when applicable, action identifier and digest allow governance joins to survive changes in operational trace context. Here, durable means a stable logical identifier; it does not imply a durable storage medium.

OpenTelemetry supplies an operational data model and sampling machinery [1]. Secure cryptographically linked audit logs long predate agent telemetry [2], and tamper-evident history trees give logarithmic-size proofs that a later log version extends an earlier one [3]. Recent governance telemetry work combines policy events, provenance and enforcement [4], while append-only telemetry storage also appears in patent literature [5]. These are substantial overlaps. This paper does not introduce hash chains, event sourcing, governance telemetry or a complete audit protocol. Its subject is the SDK boundary between local evidence acceptance and fallible operational projection.

2 Event and trust contract

The normalized JSON event is validated before use. The metadata profile excludes raw prompts, outputs, source code, tool arguments/results, credentials and authorization tokens through schemas and recursive key checks. Such restrictions reduce accidental payload capture; they are not a semantic proof that arbitrary strings contain no secrets. Nanosecond timestamps are represented as decimal strings to avoid numeric precision loss across languages.

Run and action identifiers are distinct from trace/span identifiers. If an active span exists, supplied correlation fields must agree before evidence is accepted. Propagated metadata is untrusted and confers no workload identity or authorization. The caller owns instrumentation, the schema configuration, its evidence sink, operational emitters and finalization. The SDK does not install an OpenTelemetry provider or authorize the underlying action.

The trusted boundary includes an honest event producer and, for the specified evidence behavior, the reference accumulator. A custom sink can behave differently. A callback configured as durable must honestly acknowledge committed storage and supply idempotency and recovery. The accumulator cannot observe operations that bypass instrumentation or recover source facts fabricated by a producer.

3 Acceptance and projection protocol

An accumulator owns one run, an accepted entry list, an event-ID set, a sealed flag, a completeness label, a capacity and a lock. It begins empty, unsealed and unknown. Validation and copying precede a locked acceptance operation. A run mismatch, duplicate event ID, closed accumulator or exceeded cap is rejected.

For sequence i , the next commitment is SHA-256 of the previous raw 32-byte digest, i encoded as unsigned 64-bit big-endian bytes and RFC 8785 canonical event bytes. Genesis uses 32 zero bytes. The stored entry includes its sequence, event ID, previous digest, digest and event representation. Canonicalization is an established interoperability primitive [6], and the Agent Audit Trail Internet-Draft, first posted in March 2026, already links agent audit records by SHA-256 over RFC 8785 canonical bytes [7].

When a persistence callback exists, the accumulator passes it an isolated candidate entry and requires the exact Boolean value true. Any exception or other result prevents local advancement. After acknowledgment, the local list and event-ID set advance, and isolated snapshots/return values prevent external mutation through shared object references. Without a callback, acceptance is memory-only.

The client next attempts span, log and metric projections. Each projection exception is recorded separately; it does not delete an accepted entry and does not prevent attempts of the remaining projections. The current Python implementation calls these projections sequentially. The result field named `evidence_persisted` means the selected sink returned successfully; with an in-memory accumulator it must not be read as proof of disk persistence.

4 Properties and counterexamples

The ordering property is conditional: when the reference evidence sink is configured, any operational projection attempted by an emit invocation follows successful return from its evidence append. In durable callback mode, successful append additionally follows callback acknowledgment. This follows from program order and exception propagation at the boundary. It is not an atomic transaction across the SDK, external storage and every exporter.

If an external store commits and then loses its acknowledgment, the local prefix remains unchanged and a retry may call the store again with the same event. A deliberately non-idempotent callback in our probe records two external writes. The production callback must deduplicate retries and detect conflicting bytes; the SDK does not implement a distributed exactly-once protocol or crash recovery.

Sealing closes the prefix and stores a producer-supplied completeness label: complete, incomplete or unknown. Unsealed snapshots are unknown. A separate probe seals an empty accumulator as complete successfully. Thus the label is not an independently verified property. Establishing complete capture would also require a

defined required-event set, correct instrumentation, terminal-state coverage and recovery evidence. A hash chain alone provides none of those premises.

The commitment authenticates neither the event source nor the final head by itself. A trusted checkpoint or separately signed final evidence container is needed to detect substitution or truncation relative to a known head. Optional TRACE finalization is an integration boundary, not a repair for missing source events. At the pinned revision it did not check the chain either. The finalizer took its measurement from the snapshot's chain digest and its appraisal from the snapshot's entries without recomputing one from the other, and a sealed snapshot holds mutable objects, so a snapshot with an edited entry, a dropped entry or a substituted chain digest was signed. Pull request 65 of the source repository, released in 0.1.0-alpha.5 on September 26, 2026, recomputes the chain before signing in both SDKs; its regression tests fail against the pinned code.

5 Reproducible evaluation

The source revision is agentrust-telemetry 29c13cfa4047a0f5784a90df63b22a3568453158 [8]. September 4, 2026 execution produced 111 passing Python unit tests and expected verdicts for all 13 portable fixtures, six valid and seven invalid. A rerun on September 27, 2026 against a git archive of that commit, whose embedded commit identifier was checked before extraction, produced the same counts. The fixtures cover the six event families and failures such as prohibited content, invalid trace identifiers, timestamp ambiguity, missing action digests and unsupported usage aggregation. The repository suite also exercises adapters, propagation, projection, evidence and finalization.

Four additional review probes establish: an exporter exception preserves an accepted event; callback refusal prevents projection and local advancement; an empty complete declaration is accepted; and commit-before-ack loss requires external idempotency. The last two are counterexamples to stronger claims. The probes use the actual modules at the pinned revision and record results in boundary-probes.json. All four probes gave the same result at the later revision c351568 (September 26, 2026); between the two revisions the Python evidence module changed once, to remove a deadlock on reentrant append or seal (pull request 46), while validation and TRACE finalization changed in pull request 65. Commands, source commit identifiers, dependency versions and test logs accompany the paper.

The September 27 rerun used Windows with Python 3.12.10, cryptography 50.0.0 and jsonschema 4.26.0; the September 4 run used Python 3.12.13 with the same two packages. The implementation and its repository tests share assumptions; the review probes are separate scenarios but are not an independent organizational reproduction. TypeScript schemas and golden fixtures are part of the project, but its TypeScript suite was not freshly executed in this review. No crash-restart store, multi-process correctness proof or performance benchmark is reported.

6 Relation to prior logging systems

The useful distinction is preservation of an accepted evidence prefix across operational export failures, together with refusal-before-projection when the evidence path fails. Prior write-ahead and fault-tolerant logging systems may supply closely related ordering. One example tags and checksums batches between log agents and collectors so that delivery can be confirmed [9]; its claims were not compared with this contract. Terminology and deployment in an agent system are not, by themselves, evidence of a new ordering property.

7 Limits and next experiments

The result concerns accepted event representations. It does not prove source truth, complete instrumentation, physical execution, secrecy of arbitrary metadata or authoritative policy enforcement. Next experiments should inject durable-store acknowledgment loss and restart, compare recovered prefixes, test same-ID/different-content conflicts and explicitly define completeness criteria before attempting to verify them. A deployable contract should make memory acceptance, durable acknowledgment, export success and completeness visible as distinct outcomes.

References

- [1] OpenTelemetry. Sampling. Project documentation, accessed September 27, 2026. <https://opentelemetry.io/docs/concepts/sampling/>.
- [2] B. Schneier and J. M. Kelsey (Counterpane Internet Security). Event auditing system. US Patent 5,978,475, granted November 2, 1999. <https://patents.google.com/patent/US5978475A/en>.
- [3] S. A. Crosby and D. S. Wallach. Efficient Data Structures for Tamper-Evident Logging. 18th USENIX Security Symposium, 2009. https://www.usenix.org/legacy/event/sec09/tech/full_papers/crosby.pdf.
- [4] A. Pathak and N. Jain. Governance-Aware Agent Telemetry for Closed-Loop Enforcement in Multi-Agent AI Systems. arXiv:2604.05119v1, April 6, 2026. <https://arxiv.org/abs/2604.05119v1>.
- [5] X. Ma and Z. Yin (Cisco Technology). Configuration and operational network observability using sliding window for append-only telemetry log. US Patent Application Publication 2025/0016076 A1, January 9, 2025. <https://patents.google.com/patent/US20250016076A1/en>.
- [6] A. Rundgren, B. Jordan and S. Erdtman. JSON Canonicalization Scheme (JCS). RFC 8785, June 2020. <https://www.rfc-editor.org/rfc/rfc8785>.
- [7] R. Sharif. Agent Audit Trail: A Standard Logging Format for Autonomous AI Systems. IETF Internet-Draft draft-sharif-agent-audit-trail, revision -00 posted March 29, 2026; revision -06 current on October 1, 2026. <https://datatracker.ietf.org/doc/draft-sharif-agent-audit-trail/>.
- [8] AgentTrust Telemetry source, pinned revision 29c13cfa4047a0f5784a90df63b22a3568453158. <https://github.com/agenttrust-io/agenttrust-telemetry/tree/29c13cfa4047a0f5784a90df63b22a3568453158>.
- [9] J. M. Hsieh and H. N. Robinson (Cloudera). Collecting and aggregating log data with fault tolerance. US Patent 9,081,888 B2, granted July 14, 2015. <https://patents.google.com/patent/US9081888B2/en>.